Experiment # 05

Setting up environment variables using system calls in Linux.

Objective

The learning objective of this lab is for students to understand how environment variables affect program and system behaviors

Software Tools

- Ubuntu 16.04
- GCC Complier

Theory

Environment variables are a set of dynamic named values that can affect the way running processes will behave on a computer. They are used by most operating systems since they were introduced to Unix in 1979. Although environment variables affect program behaviors, how they achieve that is not well understood by many programmers. As a result, if a program uses environment variables, but the programmer does not know that they are used, the program may have vulnerabilities.

Environment variables allow you to customize how the system works and the behavior of the applications on the system. For example, the environment variable can store information about the default text editor or browser, the path to executable files, or the system locale and keyboard layout settings.

Environment Variables and Shell Variables

Variables have the following format:

```
KEY=value

KEY="Some other value"

KEY=value1:value2
```

- The names of the variables are case-sensitive. By convention, environment variables should have UPPER CASE names.
- When assigning multiple values to the variable they must be separated by the colon : character.
- There is no space around the equals = symbol.

Variables can be classified into two main categories, environment variables, and shell variables.

Environment variables are variables that are available system-wide and are inherited by all spawned child processes and shells.

Shell variables are variables that apply only to the current shell instance. Each shell such as zsh and bash, has its own set of internal shell variables.

There are several commands available that allow you to list and set environment variables in Linux:

- env – The command allows you to run another program in a custom environment without modifying the current one. When used without an argument it will print a list of the current environment variables.
- printenv – The command prints all or the specified environment variables.
- set – The command sets or unsets shell variables. When used without an argument it will print a list of all variables including environment and shell variables, and shell functions.
- unset – The command deletes shell and environment variables.
- export – The command sets environment variables.

List Environment Variables

The most used command to displays the environment variables is printenv. If the name of the variable is passed as an argument to the command, only the value of that variable is displayed. If no argument is specified, printenv prints a list of all environment variables, one variable per line.

For example, to display the value of the HOME environment variable you would run:

```
printenv HOME
```

The output will print the path of the currently logged in user:

```
/home/linuxize
```

You can also pass more than one argument to the printenv command:

```
printenv LANG PWD

en_US

/home/linuxize
```

If you run the printenv or env command without any arguments it will show a list of all environment variables:

```
printenv
```

The output will look something like this:

```
LS_COLORS=rs=0:di=01;34:ln=01;36:mh=00:pi=40;33:so=01;35;...

LESSCLOSE=/usr/bin/lesspipe %s %s

LANG=en_US
```

```
S_COLORS=auto
XDG_SESSION_ID=5
USER=linuxize
PWD=/home/linuxize
HOME=/home/linuxize
SSH_CLIENT=192.168.121.1 34422 22
XDG_DATA_DIRS=/usr/local/share:/usr/share:/var/lib/snapd/desktop
SSH_TTY=/dev/pts/0
MAIL=/var/mail/linuxize
TERM=xterm-256color
SHELL=/bin/bash
SHLVL=1
LANGUAGE=en_US:
LOGNAME=linuxize
XDG_RUNTIME_DIR=/run/user/1000
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
LESSOPEN=| /usr/bin/lesspipe %s
_=/usr/bin/printenv
```

Below are some of the most common environment variables:

- USER - The current logged in user.
- HOME - The home directory of the current user.
- EDITOR - The default file editor to be used. This is the editor that will be used when you type edit in your terminal.
- SHELL - The path of the current user's shell, such as bash or zsh.
- LOGNAME - The name of the current user.
- PATH - A list of directories to be searched when executing commands. When you run a command the system will search those directories in this order and use the first found executable.

- LANG - The current locales settings.

- TERM - The current terminal emulation.
- MAIL - Location of where the current user's mail is stored.

The printenv and env commands print only the environment variables. If you want to get a list of all variables, including environment, shell and variables, and shell functions you can use the set command:

```
set
```

The command will display a large list of all variables, so you probably want to pipe the output to the less command.

```
set | less
```

You can also use the echo command to print a shell variable. For example, to print the value of the BASH_VERSION variable you would run:

```
echo $BASH_VERSION

4.4.19(1)-release
```

Setting Environment Variables

To better illustrate the difference between the Shell and Environment variables we'll start with setting Shell Variables and then move on to the Environment variables.

To create a new shell variable with the name MY_VAR and value Linuxize simply type:

```
MY_VAR='Linuxize'
```

You can verify that the variable is set by using either echo $MY_VAR of filtering the output of the set command with grep set | grep MY_VAR:

```
echo $MY_VAR

Linuxize
```

Use the printenv command to check whether this variable is an environment variable or not:

```
printenv MY_VAR
```

The output will be empty which tell us that the variable is not an environment variable.

You can also try to print the variable in a sub-shell and you will get an empty output.

```
bash -c 'echo $MY_VAR'
```

The export command is used to set Environment variables.

To create an environment variable simply export the shell variable as an environment variable:

```
export MY_VAR
```

You can check this by running:

```
printenv MY_VAR

Linuxize
```

If you try to print the variable in a sub-shell this time you will get the variable name printed on your terminal:

```
bash -c 'echo $MY_VAR'

Linuxize
```

You can also set environment variables in a single line:

```
export MY_NEW_VAR="My New Var"Copy
```

Environment Variables created in this way are available only in the current session. If you open a new shell or if you log out all variables will be lost.

Manipulating Environment Variables using C program

We study the commands that can be used to set and unset environment variables. Now we use multiple system calls to get or set the Environment Variables.

getenv()

The **getenv**() function searches the environment list to find the environment variable *name*, and returns a pointer to the corresponding *value* string.

Function prototype:

char * getenv(const char *name);

Function parameters:

- Name: the name of the environment variable you want to get

return value:

- The call successfully returns a pointer to value
- Call failed to return NULL

Putenv()

The **putenv** function is used to add or modify environment variables to the environment table. If there is no name environment variable in the environment table, add the environment variable; if the environment variable has name in the environment table, delete the previous value and then modify it to the new value.

Function prototype:

int putenv(char * string);

Function parameters:

- String: a pointer to an environment variable, where the environment variable must be given as "name=value"

return value:

- The call returns successfully 0
- Returns a non-zero value when the call fails

Setenv()

The **setenv** function is similar to the putenv function and can be used to add or modify environment

Function prototype:

int setenv(const char *name, const char *value, int overwrite);

Function parameters:

- Name: environment variable name
- Value: the value of the environment variable
- Overwrite: Overwrite option, when name exists in the environment table, if the value of overwrite is 0, the value of name is not modified; if the value of overwrite is non-zero, the value of name is modified; if there is no such environment variable, This environment variable is added regardless of the value of overwrite.

The environment variable of the shell process cannot be added or modified by this function, or the environment variable set by the setenv function is only valid in this process, and it is valid in this execution. If the setenv function is executed when the program is run once, the program is run again after the process terminates. The last setting is invalid, and the last set environment variable cannot be read.

return value:

- The call returns successfully 0
- Return non-zero when the call fails

Although the putenv function and the setenv function are similar in function, there are still differences in the implementation of these two functions. The differences are as follows:

Putenv function:

- The putenv function fills in the parameter string directly into the environment table, and no longer allocates memory for the string "name=value". If the string is defined in a function, after the function is called, the content pointed to by string may be released, and the value of the name

environment variable cannot be found.

Setenv function:

- The setenv function is different from the putenv function in that it copies a copy of the contents pointed to by name and value and allocates memory for it, forming a string of "name=value" and writing its address to the environment table. So there will be no case of putenv above, even if the function returns, the content pointed to by name and value is released, there is still a copy.

Example Code:

```
#include  <stdio.h>
#include <stdlib.h>
int main()
{
    char * historysize = getenv("HISTSIZE");
     printf("Original value of HISTSIZE: %s\n", historysize);

     putenv("HISTSIZE=500");
     historysize = getenv("HISTSIZE");
     printf("Changed value of HISTSIZE after getenv: %s\n", historysize);

     setenv("HISTSIZE","300",1);
     historysize = getenv("HISTSIZE");
     printf("Changed value of HISTSIZE after setenv: %s\n", historysize);

  return 0;
}
```

In this example code first get the history size and manipulate its value using putenv and set env functions.

Lab Task:

1. **Execute all mentioned shell commands to print/set environment variables.**
2. **Run above example code and change the size of history to 100.**
3. **Write a c program to get the system username using getenv() and update it to your surname using putenv() , now print the updated username and set it back to original username using setenv() system call.**

Department of Computer Engineering
HITEC University Taxila